

Whole-year parsing: return only a checked value

8 October 2026 | C003a implementation | Synthetic teaching examples

Sidiq review status: Not yet reviewed.

Research question and hypothesis

Can one small C++ function enforce yesterday's whole-year text contract before returning an integer? Hypothesis: checking spelling first, then checked conversion and complete consumption, accepts our permitted inputs without silently changing their meaning. Today introduces one concept: a function whose branches guard the value it returns. This is engineering research, not an investment result.

October 5 demonstrated lost fractional information; October 7 defined the grammar after the interrupted October 6 run. Today completes the implementation slot originally dated October 7. Boundary-oracle, failure, reproduction and consolidation lessons stay in order; we do not combine them to catch up.

Beginner explanation and terms

A parser turns text into a value under a stated contract. A branch chooses what happens when a condition is true. An exception reports failure instead of returning a normal value. A caller is the code using the function. Our caller gets a nonnegative int or an exception, never an error disguised as zero. Zero is a valid count, so returning zero for missing input would confuse two different states.

The contract accepts one or more ASCII decimal digits with value at most the target int maximum. Leading zeros are allowed. Empty text, signs, whitespace, decimal points, suffixes, embedded NUL and non-ASCII digit encodings fail. We inspect the complete string length. No trimming or rounding occurs. Spelling errors raise `std::invalid_argument`; digit-only overflow raises `std::out_of_range`. If an enormous number also has a suffix, spelling fails first. Error wording is not a stable API promise.

Mathematics and worked financial connection

For digits d_1 through d_k , start $n_0 = 0$ and define $n_j = 10 * n_{j-1} + d_j$. For 002 the intermediate values are 0, 0, 2. This explains decimal meaning; our C++ function delegates checked numeric conversion to the library instead of using a potentially overflowing accumulation in int.

For accepted $n = 2$, principal $P = 1000$ currency units and effective annual rate $r = 0.05$, $B = P * (1 + r)^n$. Year one adds 50 to reach 1050; year two adds 52.50 to reach 1102.50. The fixed C++ example obtains n by parsing 002, then calls the existing savings function. An independent Python Decimal interest ledger agrees. Text 2.5 throws and supplies no normal result.

Fixed input	Observed outcome	Meaning
0	Returns 0	Explicit zero years
002	Returns 2	Leading zeros are decimal
Empty text	<code>invalid_argument</code>	No digits supplied
2.5 or 2x	<code>invalid_argument</code>	Forbidden character
Target int maximum	Returns maximum	Inclusive representation limit
Maximum text followed by 0	<code>out_of_range</code>	Ten times maximum cannot fit

Read the small function in order

Source: `cpp/projects/savings_growth/years.cpp`, with its declaration in `years.hpp`. The header's include guard prevents duplicate declarations. `string` provides text, `charconv` provides conversion, `stdexcept` provides exceptions, and `system_error` names conversion error codes. These are standard library facilities; no dependency is added. The following explanation follows each executable line.

1. `int parse_whole_years(const std::string& text)` declares an integer result and a read-only reference to the caller's string. The ampersand avoids copying that string; `const` prevents modification through this parameter. The caller's string stays alive during the call, and the function retains no reference.
2. `if (text.empty())` tests whether there are no characters. The braces group its branch. `throw std::invalid_argument(...)` stops normal execution; the final `return` is not reached for empty text.
3. `for (const char character : text)` visits every character, including an embedded NUL. This is a range-based loop, not a vector. Each iteration names one character without changing the string.
4. The inner `if` asks whether character is below '0' or above '9'. Single quotes denote characters; the logical OR operator means either condition is enough. A forbidden character throws immediately. This rejects -0 as spelling, even though a generic signed conversion might produce numeric zero.
5. `int years = 0` initializes local storage. Initialization is not a fallback result: subsequent checks must succeed before this value can leave the function. Failure never returns the initialized zero.
6. `const char* end` stores the position just after the input, calculated with `data() + size()`. A pointer identifies a position in the string; this one is a boundary, not a character to read. The earlier empty check ensures conversion is applied only after at least one digit has been found.
7. `const auto result = std::from_chars(text.data(), end, years, 10)` attempts base-ten conversion. `auto` asks the compiler to infer the result type. The library writes through its `years` argument and returns an error code `ec` and stopping pointer `ptr`. No floating-point intermediate is used.
8. If `ec` equals `std::errc::result_out_of_range`, throw `std::out_of_range`. This handles an otherwise valid spelling that cannot fit the receiving `int`. There is no unsafe `INT_MAX + 1` computation.
9. The next `if` rejects any other conversion error or a stopping pointer unequal to `end`. The explicit complete-consumption check documents the return condition even though the preceding digit scan has already ruled out suffixes. With conforming conversion it is a defensive check, not a separate observed failure path. We do not claim branch coverage for it.
10. `return years` is reached only after all checks pass. The function prints nothing, so the same implementation serves the fixed demonstration and tests. It leaves financial validation to C002.

Catching failure at the caller

In `parse_years_demo.cpp`, a `try` block attempts the fixed fractional input. A matching `catch` handles `invalid_argument` and prints rejection. If parsing unexpectedly returns, the demo exits with failure. The separate test harness catches the required exception type; a missing or wrong exception fails the run. It uses `exit` status instead of `assert`, so disabling assertions cannot disable these checks.

Evaluation protocol, implementation and results

Synthetic strings have no market source or return period. The predeclared checks are exact values, required errors, a hand balance, repeatable output and regression checks. No parameters were fitted.

Added years.hpp/cpp, test_years.cpp, parse_years_demo.cpp and hand-authored expected output. Make runs both new executables. C002 is unchanged; a complete command-line interface remains deferred.

Check	Observed result
Parser harness	30 checks pass: 8 accepted, 21 rejected, 1 savings connection
Long text cases	1000 leading zeros then 2 accepted; 1000 nines rejected
New demo replay	Two runs match each other and the hand-authored expected bytes
Independent financial arithmetic	Decimal interest ledger gives 1050 then 1102.50
Full make check	608 Python tests, 66 prior C++ checks, 30 new checks, four output fixtures pass
Python quality	Ruff lint, 49 formatted files and strict mypy on 21 source files pass
Fresh build and dependencies	Six warning-clean C++17 executables; pip consistency passes

Reproduce from the repository root: make cpp-parse-years; make check; make -B cpp-check;

.venv/bin/python -m pip --no-cache-dir check. Render this note with make note

NOTE_DATE=2026-10-08. Logs are in work/2026-10-08. Checks passed first run. The independent Python boundary oracle is next.

Assumptions, costs and limitations

Savings assumes fixed-rate reinvestment without cash flows, fees, tax or inflation. Costs would reduce wealth; none is estimated. No empirical return or profitability is claimed.

C002 can still reject balance overflow from accepted years. Representability is not economic realism. The parser accepts a complete std::string; if a caller truncates bytes before constructing that string, this function cannot recover them. The NUL tests use an explicit length to preserve the entire input.

Work grows with input length; there is no length cap. External resource protection remains future work, and resource failures can raise other exceptions. No security, coverage or speed claim is made. The digit comparison assumes an ASCII-compatible execution character set, as on the current host and CI targets. Sidiq's personal understanding is not inferred from test success.

Primary sources consulted on 8 October

Microsoft's from_chars reference documents conversion error codes, representability and stopping position. Our digit-only grammar and error precedence remain project policy.

<https://learn.microsoft.com/en-us/cpp/standard-library/charconv-functions?view=msvc-170>

Microsoft's exception reference explains try, throw and catch control flow. We reuse the project's exception style to keep an invalid count separate from a normal returned value.

<https://learn.microsoft.com/en-us/cpp/cpp/try-throw-and-catch-statements-cpp?view=msvc-170>

Interview explanation

I defined spelling and range before implementation. The parser rejects empty or non-digit strings, then checks the library conversion's error and end position before returning an int. Errors throw instead of returning zero, because zero years is valid. Fixed expected values and required exceptions check the contract; a separate hand ledger checks the financial connection. This demonstrates input validation mechanics, not that a savings model predicts investment returns.

Progress, cadence and next task

The parser implementation slot is complete on October 8. C003a remains in progress until independent boundary verification, the deliberate failure lesson, reproduction and consolidation are complete. Next is the boundary-oracle slot, now October 9; subsequent prerequisites move in order. The complete CLI, vectors/statistics and October 12 ledger remain conditional. We do not compress the curriculum.

Today's shared Europe/London draw is 3. History and daily/weekly memories show zero programme commits today before work. One coherent lesson is ready and there is no pending unit to integrate, so plan one commit and leave two slots unused. Publication results and final actual count belong in the run memory and briefing after the push; they are not invented here. Personal review remains Not yet reviewed.

No input is required for the next synthetic lesson. Q004a lawful source access/sample evidence and Q005b integration remain gates before empirical research. The daily schedule remains 08:00 Europe/London, including daylight-saving changes. Stop new research after 31 December 2026 and request the next programme scope. Existing publication authority does not imply personal review.

Check your understanding

1. Why throw for empty text instead of returning zero?
2. Walk through 002 and 2.5. Which statement returns a value, and which branch rejects?
3. Why inspect original characters if we will also call a numeric conversion function?
4. What happens when a digit-only value is too large for int? Does an accepted int guarantee that the savings calculation succeeds?
5. What does `const std::string&` mean, and why must an embedded NUL input retain its length?

Suggested answers

1. Empty text supplies no count; 0 explicitly requests zero years. Returning the same ordinary value for both would hide missing input. Throwing keeps failure separate from a valid result.
2. Every character of 002 is a digit; decimal conversion succeeds and consumes the whole range; return years gives 2. The dot in 2.5 triggers `invalid_argument` during the scan, before conversion.
3. A library's numeric grammar differs from our model's policy. A sign or readable numeric prefix must not become acceptable merely because a number can be extracted. Checking bytes enforces spelling.
4. Conversion reports `result_out_of_range` and the wrapper throws `out_of_range`. A representable count can still overflow a positive-rate balance; the financial function keeps its own validation.
5. It is a read-only reference to an existing string, avoiding a copy. An embedded NUL is a byte inside that string, not its end. Explicit-length construction preserves bytes that a C-string constructor would discard after the first NUL; the parser checks only the input it actually receives.